

Perché modellare un processo?

Dare ordine, controllo e ripetibilità!

- Processo introduce organizzazione, stabilità, controllo, in attività che, lasciata libera, può diventare piuttosto caotica ...
- Con l'intenzione di migliorare:
 - la produttività degli sviluppatori
 - qualità del SW prodotto

Code and Fix

Si arriva al codice finale “per tentativi”

- Non si eseguono le fasi di analisi e progettazione
- Può essere usato in progetti piccoli
- LOCs < 1500
- Si è dimostrato non adatto per progetti grandi con + sviluppatori

Modello a cascata

Storicamente il primo modello di processo di sviluppo software

- Anni '50 per grandi sistemi di difesa (US)
- Prima presentazione: 29 giugno 1956
- Prima definizione formale: articolo 1970
- Winston Royce
- Diffusione negli anni '70-'80
- Mette in cascata i passi tradizionali dello sviluppo di un sistema (software)
- Deriva dal processo manifatturiero e ha forti analogie con lo sviluppo di altri sistemi (es. costruzione di una casa/nave)
- Ogni fase produce un prodotto che è l'input della fase successiva
- Con il modello Waterfall si ha il passaggio dalla dimensione artigianale (code and fix) alla produzione industriale del software

Waterfall pregi

Pregi

- Enfasi su aspetti come l'analisi dei requisiti e il progetto di sistema trascurati nell'approccio code & fix
- Pospone l'implementazione dopo avere capito i bisogni del cliente (obiettivi)
- Introduce disciplina e pianificazione (sviluppo a fasi)
- Applicabile se i requisiti sono chiari e stabili

Difetti

- Lineare, rigido, monolitico
- no feedback tra fasi
- no sviluppo parallelo
- unica data di consegna!
- La consegna avviene dopo anni, intanto i requisiti cambiano o si chiariscono: così viene consegnato software obsoleto
- Viene prodotta molta documentazione
- L'utente spesso non conosce tutti i requisiti all'inizio dello sviluppo

Alcuni difetti superati da:

- Modello waterfall con feedback e iterazioni

Varianti del modello a cascata

- Cascata con prototipazione
- Prima di iniziare lo sviluppo del sistema vero e proprio si costruisce velocemente un prototipo "usa e getta" con il solo scopo di fornire agli utenti una base concreta per meglio definire i requisiti
- Cascata con feedback e iterazioni

V model

- Esplicita certe iterazioni che sono nascoste nel Waterfall
- Enfasi sulle fasi di testing
- Evidenzia come le attività di testing (parte destra della V) sono collegate a quelle di analisi e progettazione (parte sinistra della V)
- Ogni controllo fatto a destra che non dia buon esito porta a un rifacimento/modifica di quanto fatto a sinistra

Waterfall, V-Model: Problemi!

- Progetti reali si conformano raramente allo schema sequenziale
- Il cliente può non sapere esattamente cosa vuole all'inizio dello sviluppo
- I requisiti non si possono congelare
- Cambiano con il tempo!!!
- Il cliente deve essere paziente
- Versione funzionante solo alla fine!
- Errore in fase iniziale può avere conseguenze disastrose 17 Modelli + flessibili capaci di 'accettare' il cambiamento dei requisiti
- Nascita di modelli evolutivi per superare (parte) di questi problemi

Modelli evolutivi

- Idea: sviluppare un'implementazione iniziale, esporla agli utenti e raffinarla attraverso successivi rilasci del SW (release)
- Sottocategorie:
 1. Prototyping (prototipazione)
 2. Modelli incrementali
 3. Modelli iterativi

Modelli a prototyping

Realizzazione di un prototipo funzionante del sistema, su cui validare i requisiti (o l'architettura)

- Di solito prototipo “usa e getta”
- Altrimenti il prototipo “evolve” nel sistema vero e proprio
- Il prototipo è ridotto nel senso che
 - ha meno funzionalità a livello quantitativo e/o qualitativo
 - ad esempio, rispetto a requisiti di usabilità, di sicurezza, ...
 - è meno efficiente in termini di prestazioni
- Spesso sviluppato in un linguaggio di alto/altissimo livello
- accorcia i tempi di realizzazione

Modelli iterativi-incrementali

- Sviluppo di varie release, di cui solo l'ultima è completa (Risposta rapida al mercato!)
- Richiedono di individuare le funzionalità da rilasciare al cliente per ogni release
- Dopo la prima release, si procede in parallelo: utilizzo di una release e sviluppo della successiva (con feedback)

Modello incrementale

- Ogni release aggiunge nuove funzionalità

Sulla pianificazione

- Nei modelli incrementali è molto importante la pianificazione
- Fase nella quale si decide quali funzionalità allocare nell'iterazione successiva
- Vengono usati dei criteri di priorità tra funzionalità per stabilire quali considerare prima e quali rimandare
- Due possibili strategie:
 - Si trattano per prime le funzionalità ad alto rischio
 - Si cerca di massimizzare il valore per gli utenti cercando di sviluppare prima le funzionalità più usate

Modello iterativo

- Da subito sono presenti tutte (o buona parte del)le funzionalità che sono via via raffinate, migliorate

Modello a spirale

- Correntemente è l'approccio più realistico per sistemi di grandi dimensioni e per sistemi complessi
- Approccio "evolutivo" con interazioni continue fra cliente e developer
- Rilascio di software parziale
- Modello "risk driven": tutte le scelte sono basate sui risultati dell'analisi dei rischi
- Primo modello che considera il rischio!
- Deve essere completato/istanziato con altri modelli nella fase di Engineering
- Di fatto è un 'meta-modello'
- Es. Requisiti chiari e stabili: Waterfall Requisiti confusi: prototipo

Cos'è un rischio?

- Un rischio in ambito Software è una circostanza potenzialmente avversa in grado di pregiudicare lo sviluppo e la qualità del software
- Ogni scelta/decisione => un rischio
- Esempi: • Uso la tecnologia 'XYZ' non consolidata (è adatta? è matura?)
- Acquisto un componente software (difetti? è adatto?)
- Scelgo il framework di sviluppo 'ZZZZ' (personale adatto?)
- Aggiungo un nuovo sviluppatore al progetto (migliora?)
- Due caratteristiche importanti nella valutazione di un rischio:
 - Gravità delle conseguenze
 - Probabilità che si verifichi le circostanze

Modello a spirale

- Planning: determinazione di obiettivi, alternative, vincoli
- Risk analysis: analisi delle alternative e identificazione/risoluzione dei rischi
- Engineering: sviluppo del prodotto di successivo livello
- Customer evaluation: valutazione dei risultati dell'engineering dal punto di vista del cliente

Fasi del modello a spirale

Planning ' Identifica gli obbiettivi in termini di qualità da ottenere

Es. Sviluppo di 'n' requisiti, con prestazioni 'xyz' '

Identifica le alternative '

Es. Comprare componenti o sviluppo "from scratch"?

Identifica i vincoli imposti dalle alternative

Es. Se uso la libreria 'xyz' devo usare il linguaggio Java

Risk analysis

Identifica i rischi, valutali e prendi le precauzioni per ridurli

Es. Requisiti non chiari, usa un prototipo

Si può anche cancellare il progetto se troppo rischioso (no-go!)

Engineering ' Si sceglie un modello e si sviluppa

Es. Requisiti chiari e stabili: waterfall

Requisiti ancora confusi: altro prototipo

Customer evaluation

Revisione dei risultati degli altri tre passi con il cliente

UP

Unificato (1996): proviene dalla storia della sua nascita: ha unificato i metodi di sviluppo proposti da Booch, Rumbaugh e Jacobson (i tres amigos di UML)

- Primo libro: The Unified Software Development Process published in 1999 by Jacobson, Booch and Rumbaugh
- Sviluppo di sistemi ad oggetti, con uso di Notazione UML per tutto il processo
- Guidato dagli Use Case
- Incorpora molte delle idee 'buone' dal Modello a spirale
- sviluppo a fasi, risk analysis, rilasci multipli, enfasi su customer evaluation
- Schema di modello da adattare/personalizzare ai diversi casi e realtà aziendali
- Meta-modello
- Supportato da tool (visuali) in ogni fase

UP le fasi

- Possibili diverse iterazioni che terminano con il rilascio del prodotto
- Ogni iterazione consiste di quattro fasi (anche ripetute + volte) che terminano con una milestone
- Ogni fase è costituita da diverse attività • Requisiti (R), Analisi (A), Design, (D) Codifica (C) e Testing (T)

UP le fasi

Inception (avvio)

- Studio di fattibilità
- Requisiti essenziali del Sistema
- Risk analysis

Elaboration

Sviluppa:

- la comprensione del dominio e del problema
- gli Use case della release da rilasciare
- l'architettura del Sistema

Construction

- Design (in UML)
- Codifica e testing del Sistema

Transition

- Messa in esercizio della release nel suo ambiente (deploy)
- Training e testing da parte di utenti fidati

Sviluppo_basato_sui_componenti

Sistemi software prodotti per composizione (metafora dei 'Legò')

- Componenti software preconfezionati (commerciali e non)
- Modello che va nella direzione del riutilizzo del software
- Esiste uno 'Store' o 'Marketplace' dei componenti
- Lo sviluppo inizia con i requisiti e l'identificazione dei componenti.

Metodi plan driven vs metodi agili

Metodi plan-driven (+prescrittivi)

- ☐ Basati su idea ingegneristica
- ☐ Qualità del processo => Qualità del prodotto
- ☐ Seguono un approccio classico dell'ingegneria dei sistemi fondato su processi ben definiti e con i passi standard
- ☐ Requisiti, Progetto, Codifica, Testing
- ☐ Importanza della documentazione (completa)
- ☐ Esempi: Waterfall, V-Model, modello a spirale, UP

Metodi agili (+adattivi)

- Internet economy chiede flessibilità e velocità
- Rispondere ai cambiamenti in modo veloce
- Filosofia del programmare come “arte” (craft) piuttosto che processo industriale
- Cosa più importante soddisfare il cliente e non seguire un piano (contratto)
- Vari approcci anche molto diversi tra loro: Extreme Programming (XP), Crystal, Scrum

Ciascuno degli approcci si trova a proprio agio se “gioca in casa”

- Metodi plan-driven adatti con:
 - Sistemi grandi e complessi, safety-critical o con forti richieste di affidabilità
 - Requisiti stabili e ambiente predicibile
- Metodi agili adatti con:
 - Sistemi e team piccoli, clienti e utenti disponibili, ambiente e requisiti volatili (che cambiano rapidamente)
 - Team con molta esperienza
 - Tempi di consegna rapidi

Dev ops fasi

Si definiscono i requisiti, gli obiettivi e le funzionalità del software o della nuova versione. È la fase di ideazione e definizione del progetto.

Gli sviluppatori scrivono il codice del software, utilizzando sistemi di controllo versione (come Git) ed inviano il codice su un server remoto condiviso (come GitHub)

Fase in cui l'app viene assemblata in automatico con le librerie e viene prodotto un eseguibile (es. Jar file)

Si eseguono test automatici e manuali per verificare la qualità del software e assicurarsi che soddisfino i requisiti.

Il software che ha superato i test viene distribuito nell'ambiente di produzione (l'ambiente reale dove gli utenti lo useranno)

Il software viene eseguito in produzione. Il team Operations si assicura che l'infrastruttura sia stabile e che il servizio sia sempre disponibile (scaling e load balancing)

Si raccolgono dati sulle prestazioni dell'applicazione (es. utilizzo della CPU, errori, tempi di risposta) e sull'esperienza utente. Questo è fondamentale per identificare problemi e aree di miglioramento

Continuous integration

La Continuous Integration (CI), o Integrazione Continua, è una pratica di sviluppo software in cui i programmatori integrano frequentemente il proprio lavoro (codice) nel repository condiviso del progetto, in genere diverse volte al giorno